

Malware Disassembly and Code Analysis Using Ghidra

EXPERIMENT NO: 10

DATE:

1. AIM

To set up a secure analysis environment, utilize the Ghidra reverse engineering framework to disassemble and decompile binary code, and interpret low-level assembly functions to identify common malware behaviors, persistence mechanisms, and indicators of compromise.

2. SOFTWARE / TOOLS REQUIRED

- **Operating System:** Isolated Virtual Machine (Windows/Linux OS)
- **Forensics Tool:** Ghidra (Open-source software reverse engineering suite developed by the NSA)
- **Runtime Environment:** Java Development Kit (JDK) required for running Ghidra
- **Sample Files:** Benign sample binaries (benign_sample1.bin, benign_sample2.hex) or controlled test binaries exhibiting typical executable structures.

3. DESCRIPTION

Malware analysis is the critical process of dissecting compiled software to understand its capabilities, origins, and operational intent without executing it live (static analysis). Ghidra, an open-source Software Reverse Engineering framework developed by the National Security Agency (NSA), is utilized extensively by forensic investigators for this purpose. When original source code is unavailable or intentionally obfuscated by adversaries, analysts rely on Ghidra to bridge the gap between raw, machine-executable binaries and human-readable logic.

The framework's core strength lies in its CodeBrowser, which parses raw binary instructions into low-level assembly language, while its advanced Decompiler algorithmically reconstructs higher-level, C-like pseudo-code. This dual approach significantly reduces the cognitive load required to decipher complex malware algorithms. To further aid in understanding the execution logic, Ghidra generates interactive Control Flow Graphs (CFGs) that visually map out function calls, decision branches, and loops, allowing analysts to trace execution paths and identify evasion techniques.

4. PROCEDURE

Step 1: Installation, Project Initialization, and Binary Import

1. Open the Kali Linux terminal and install the Ghidra framework using the advanced packaging tool:

Bash: `sudo apt update && sudo apt install ghidra -y`

2. Launch Ghidra from the terminal by typing `ghidra`
3. create a new non-shared project titled `Ghidra_by_Ragul`.
4. Navigate to `File > Import File`, select the target binary (`/bin/ls`), and proceed with the default import configuration.

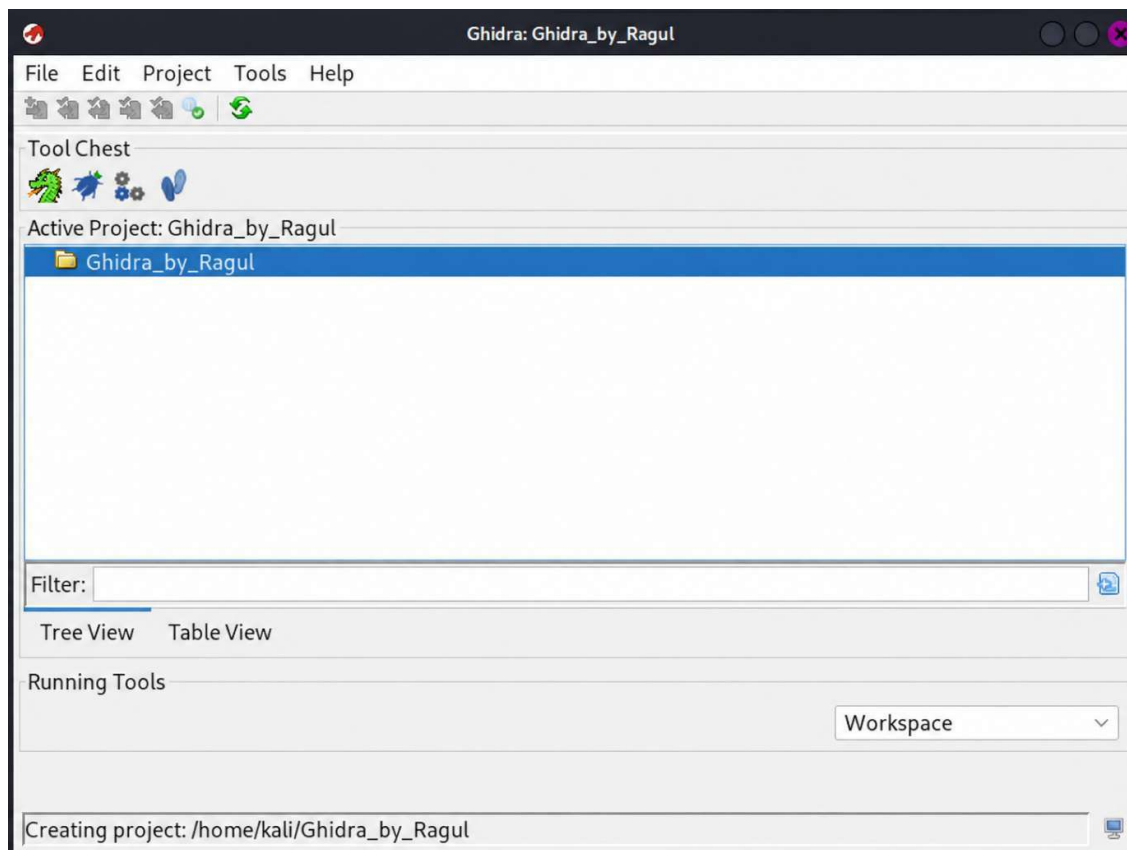


Figure 1: Initial Ghidra Project Window showing the newly imported binary

4. Double-click the imported file to open it in the CodeBrowser tool and execute the initial Auto-Analyze function when prompted.

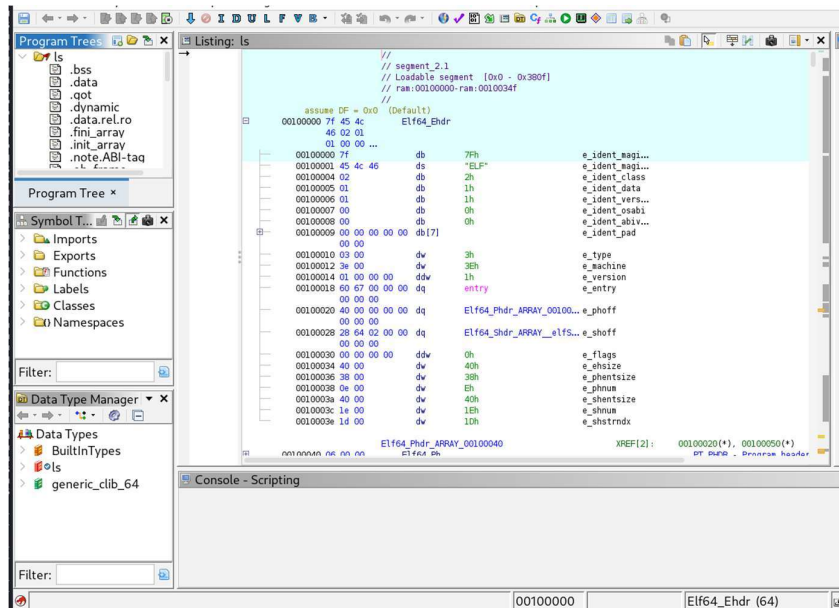


Figure 2: Ghidra CodeBrowser interface displaying the disassembly after Auto-Analysis

Step 2: String Extraction and Import Analysis

1. Navigate to Window > Defined Strings to extract hardcoded ASCII and Unicode strings from the binary.
2. In the Symbol Tree pane on the left, expand the Imports directory to identify the external shared libraries the binary relies on (e.g., libc.so.6).

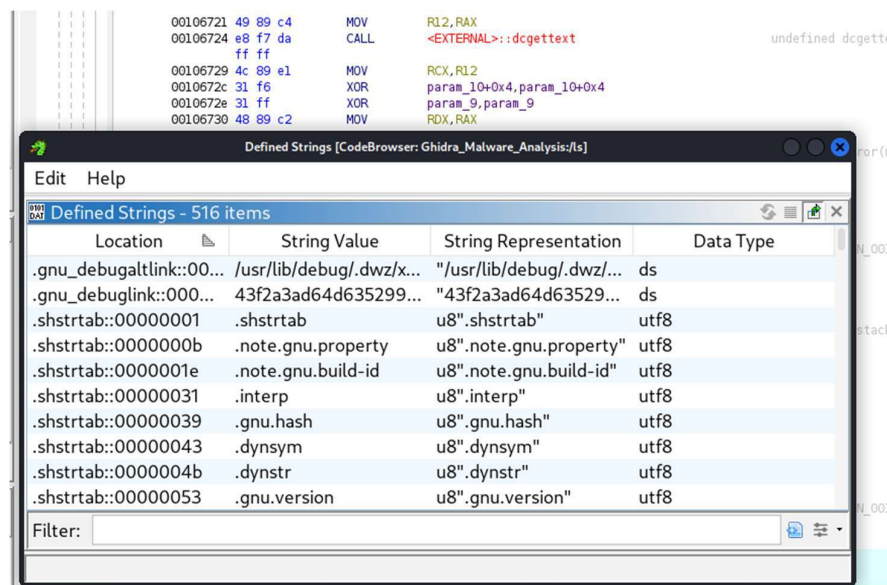
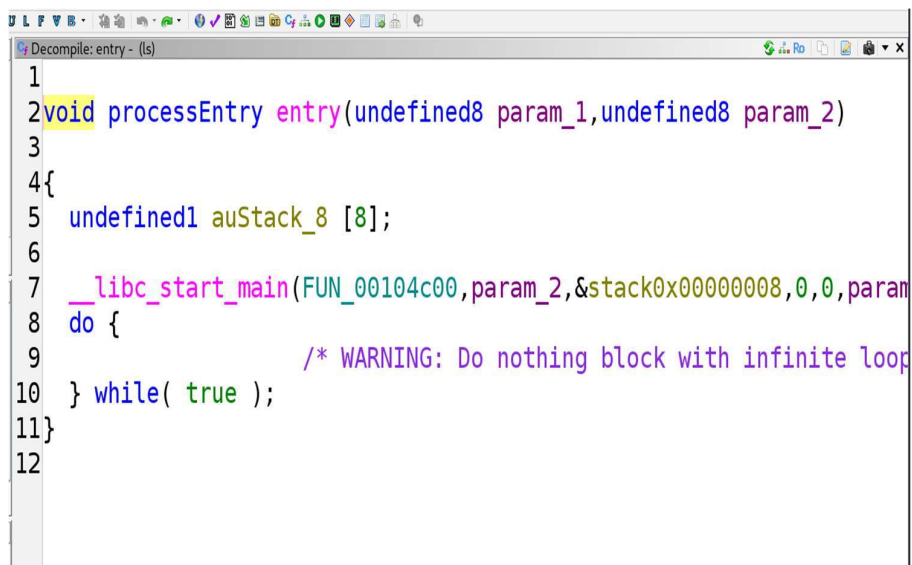


Figure 3: Extraction of Defined Strings and identification of system Imports

Step 3: Function Decompile

1. In the Symbol Tree pane, expand the Functions directory.
2. Locate and select the entry function.
3. Observe the raw Assembly instructions in the center Listing window, and analyze the translated C-like pseudocode in the right-hand Decompile window.



The screenshot shows a decompiler window titled "Decompile: entry - (ls)". The code is as follows:

```
1
2 void processEntry entry(undefined8 param_1,undefined8 param_2)
3
4 {
5     undefined1 auStack_8 [8];
6
7     __libc_start_main(FUN_00104c00,param_2,&stack0x00000008,0,0,param
8 do {
9                                     /* WARNING: Do nothing block with infinite loop
10 } while( true );
11}
12
```

Figure 4: Decompiled C-like pseudocode of the entry function

Step 4: Execution Flow Visualization (Main Function)

1. Due to the binary being stripped, the original main function name was removed. In the Decompiler window for the entry function, identify the first parameter passed to `__libc_start_main` (e.g., `FUN_00104c00`).
2. Double-click `FUN_00104c00` to navigate to the actual main execution logic of the program.
3. With `FUN_00104c00` selected, navigate to **Window > Function Graph**.
4. Analyze the visual block diagram that illustrates the complex logical execution paths, decision branches (green and red arrows), and loops within the core code.

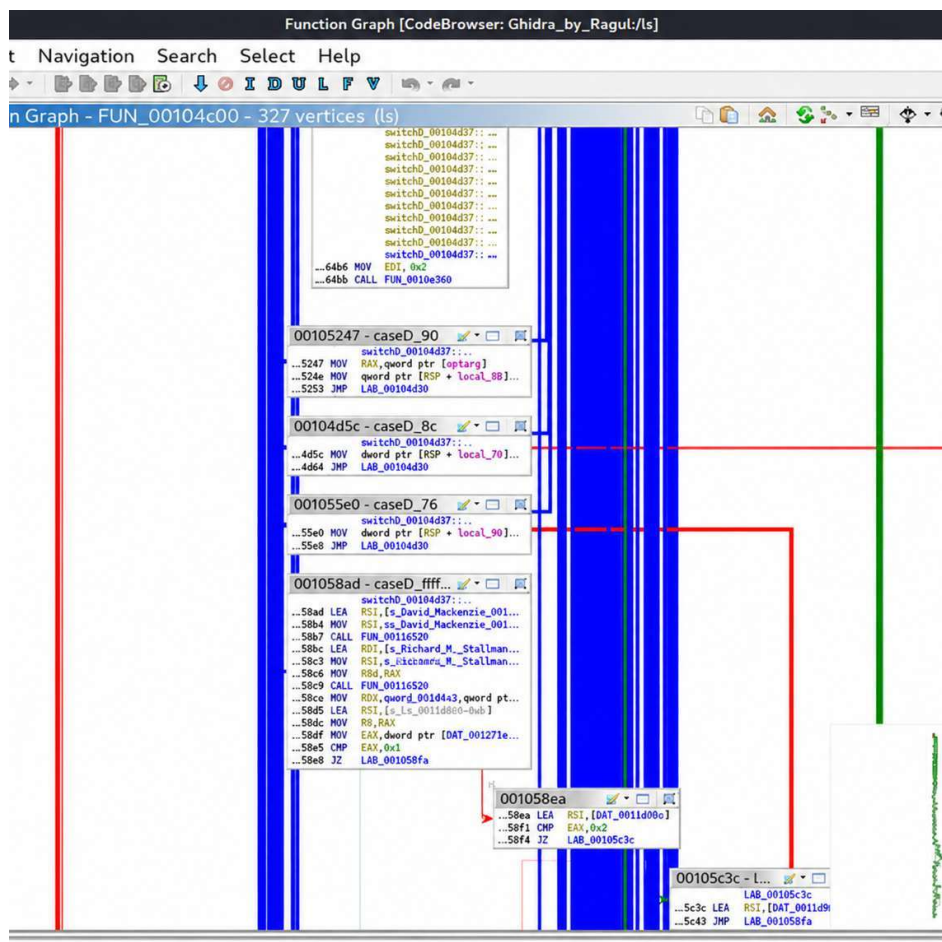


Figure 5: Visual Function Graph illustrating the complex execution flow of the stripped main function (FUN_00104c00)

5. OBSERVATIONS / FORENSIC FINDINGS

- **Code Structure:** Ghidra successfully disassembled the ELF binary, revealing the core architecture, data sections, and metadata headers.
- **Strings & Imports:** The extraction of defined strings and imports confirmed the binary's reliance on standard system libraries (libc.so.6), consistent with a benign Linux utility. In a malware context, this step would reveal imported networking, evasion, or encryption libraries.
- **Decompilation:** The Decompiler successfully translated the low-level assembly of the entry function into an understandable C-code format, highlighting the `__libc_start_main` execution call.

- **Flow Analysis:** The Function Graph provided a clear, node-based visualization of the function's structural logic, simplifying the analysis of complex execution paths and potential infinite loops.
- **Stripped Binary Analysis & Symbol Resolution:** The binary had its symbol table stripped (.symtab removed), obscuring original developer function identifiers and variable names. Forensic examination of the execution entry vector resolved the actual main routine (FUN_00104c00) by tracing the primary pointer passed into the __libc_start_main parameter list.

RUBRICS

Criteria & Marks Assigned	Mark Allotted	Mark Awarded
1. GitHub Activity & Submission Regularity	3	
2. Application of Forensic Tools & Practical Execution	3	
3. Documentation & Reporting	2	
4. Engagement, Problem-Solving & Team Collaboration	2	
Total	10	

6. RESULT

Ghidra was successfully utilized to disassemble, decompile, and analyze the target binary code. Through automated analysis, string inspection, API import tracking, and decompiler evaluation, the binary's underlying structure and behavioral logic were effectively interpreted.